SpatialHarness: Test-Time Spatial Scaffolding for Fine Robotic Manipulation

Jiayu Wang[1] Yue Yu[1] Bin Zhu[2] Zhiyao Yang[3] Jingjing Chen[3†]
[1]College of Computer Science and Artificial Intelligence, Fudan University
[2]Singapore Management University
[3]Institute of Trustworthy Embodied AI, Fudan University
{jiayuwang25,yuy24}@m.fudan.edu.cn
binzhu@smu.edu.sg chenjingjing@fudan.edu.cn
[†]Corresponding author

Abstract

Frontier multimodal foundation models (e.g., GPT-6 Astra) have recently shown strong potential for direct robotic control, yet their performance on fine manipulation remains limited. We argue that an important source of failure is not necessarily insufficient policy capability, but insufficient spatial observability, where task-critical spatial relationships may be poorly revealed by the existing physical camera setup. We introduce SpatialHarness, a test-time embodied harness that provides test-time spatial scaffolding for fine robotic manipulation without policy fine-tuning or changes to the physical sensing setup. SpatialHarness maintains an online simulated scene synchronized with real-world execution, identifies task-critical spatial relationships, and renders complementary virtual views that expose them to a frozen multimodal policy. To keep the simulated scene aligned during interaction, we develop interaction-aware scene synchronization that distinguishes static, held, and transition modes. We evaluate SpatialHarness on four real-robot manipulation tasks spanning precise geometric alignment, object-relative placement, and articulated-object interaction. Using the same frozen GPT-6 Astra policy, SpatialHarness substantially improves task success, including from 26.7% to 66.7% on plug insertion and from 0% to 100% on Tower of Hanoi. These results indicate that improving spatial observability at test time can unlock fine-manipulation capabilities already present in strong multimodal foundation models. Project website: https://emilia113.github.io/SpatialHarness/.

1. Introduction

Recent frontier multimodal foundation models, such as GPT-6 Astra [27], have begun to exhibit surprisingly strong capabilities for robotic manipulation. Given visual observations and language instructions, these models can directly generate robot actions and refine subsequent decisions through closed-loop interaction, without task-specific policy training. This emerging paradigm suggests a compelling alternative to robotic foundation models [4, 9, 15, 19, 32]. Instead of adapting the model to each robotic setting, an embodied harness can instead structure how the model observes, reasons about, and interacts with the physical world.

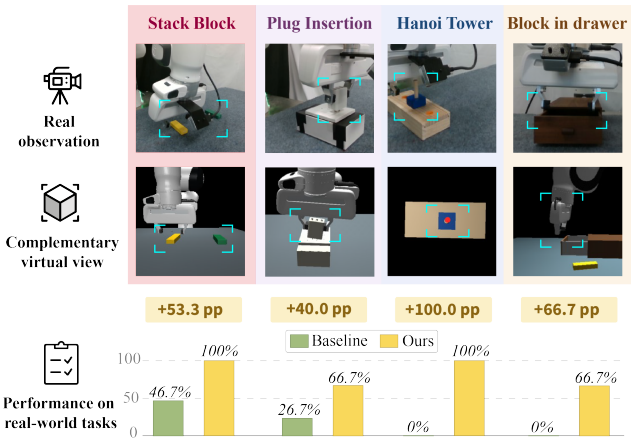


Figure 1. The top two rows show real and auxiliary virtual views, respectively, for the four tasks shown; cyan marks task-relevant regions. Below, baseline (green) and SpatialHarness (yellow) success rates are compared using the same GPT Astra Low-based manipulation policy and real camera configuration. Values above the bars show gains over the baseline in percentage points (pp).

Existing harnesses have explored action interfaces, demonstrations, interaction histories, and execution feedback to better expose the capabilities already present in general-purpose multimodal foundation models [7, 12, 23]. Nevertheless, strong high-level reasoning does not necessarily translate into reliable fine robotic manipulation [21, 31]. Tasks such as insertion, stacking, or placement onto

narrow structures depend on task- critical spatial relationships, whether a plug is aligned with its socket, whether a hole is centered over a peg, or whether a grasped object is correctly positioned relative to a supporting surface. These relationships can be difficult to infer from the fixed cameras available in a real deployment, particularly near contact, where the gripper and manipulated objects frequently occlude the most relevant geometry [25]. In such cases, failure may arise not because the policy lacks the necessary manipulation capability, but because the spatial evidence required to exercise that capability is poorly observable. This observation motivates us to ask a critical question: *How can we improve the spatial observability of a robot's environment at test time, without retraining the policy or modifying the physical sensing setup?*

To address this question, we introduce *SpatialHarness*, a test-time embodied harness that provides test-time spatial scaffolding for fine robotic manipulation. SpatialHarness maintains an online simulated scene synchronized with real-world execution and uses it to construct task-relevant spatial context, without the need to modify the policy or add extra physical cameras, as illustrated in Figure 1. Before execution, the multimodal foundation model analyzes the task goal and identifies task-critical spatial relationships. SpatialHarness then configures virtual viewpoints that expose these relationships, complementing the observations from the robot's physical cameras. During execution, the harness renders the complementary virtual views from the continually updated simulated scene and supplies them to the policy together with structured spatial information, including object 6-DoF poses and estimates of rendering reliability. A key challenge is maintaining sufficiently accurate spatial context as physical interaction changes the scene. Pure physics rollout is unreliable during manipulation because errors in physical parameters and contact modeling can cause simulated object states to diverge from real execution, particularly during grasping and release [5, 20]. SpatialHarness therefore performs interaction-aware online scene synchronization, combining visual correction grounded in real observations with programmatic correction. We distinguish three object–gripper interaction modes, including static, held, and transition, and exploit different physical and observational priors in each case. Static objects preserve or visually update their aligned states, while objects under sustained held are propagated using robot action and programmatically validated against real observations and gripper geometry. In transition mode, small pre-contact errors can substantially alter simulated outcomes. SpatialHarness performs hypothesis replay from a pre-interaction checkpoint using candidate poses and measured robot motion, then selects a corrected state through interaction consistency filtering and contour matching. Consequently, SpatialHarness uses simulation as a runtime spatial interface: real observations ground the scene, while complementary virtual views expose task-critical spatial relationships that may be hidden, ambiguous, or poorly conditioned in the physical camera configuration. physical camera observations. We evaluate SpatialHarness on four real-robot manipulation tasks spanning local geometric alignment, object-relative placement, and articulated-object interaction. Using the same frozen GPT-6 Astra policy for both the baseline and SpatialHarness, we observe substantial improvements in task success, for instance, the success rate for plug insertion increases from 26.7% to 66.7% , and 0% to 100% on Tower of Hanoi, as shown in Fig. 1. These improvements are achieved without policy fine-tuning or changes to the physical camera setup, highlighting the importance of spatial observability as a test-time factor in fine robotic manipulation.

Our contributions are as follows:

- We introduce SpatialHarness and formulate test-time spatial scaffolding as a mechanism for improving fine manipulation with frozen multimodal policies. SpatialHarness augments runtime observations with task-relevant virtual views and structured spatial context, without modifying the underlying policy or physical sensing setup.
- We develop an interaction-aware online scene synchronization framework that combines visual and programmatic correction across static, held, and transition modes to maintain alignment between the simulated scene and the real environment during manipulation.
- We demonstrate on real-robot fine-manipulation tasks that improving spatial observability can substantially increase the success rate for a strong multimodal foundation model used as a robot policy.

2. Related Work

2.1. Frontier Multimodal Models and Embodied Harnesses

Vision-language-action models such as RT-2 [32], OpenVLA [19], and π_0 [4] use robot data to adapt pretrained vision-language knowledge for action generation. As general-purpose foundation models advance [2, 8, 17, 28], frontier closed-source multimodal models have demonstrated the ability to perform robotic manipulation directly by generating actions from visual observations and adjusting them based on execution feedback. To support this direct use of general-purpose models, embodied harnesses provide interaction interfaces and structured context that help models apply their existing capabilities to specific robotic tasks. Show-Harness [7] provides a fine-grained action interface with clear semantics. RoboDawn [12] and RoboICL [23] structure demonstrations and experience gathered during interaction to help models adapt to tasks. These methods structure action interfaces and interaction

context to support robotic manipulation.

2.2. Real-to-Sim and Digital Twins

Real-to-sim research constructs simulation environments from real-world observations for robot learning and interaction. NeRF [26] and 3D Gaussian Splatting [18] provide the basis for reconstructing renderable scene representations and synthesizing novel views, while Re³Sim [13] and ARSTAG [22] further construct simulation scenes and task-relevant assets offline from real-world observations for demonstration generation and policy learning. Beyond offline learning, reconstructed scenes are also used during deployment for planning, state tracking, and policy observations. Simify [16] reconstructs simulation-ready assets from a single RGB-D observation and uses evolutionary search with parallel physics rollouts at test time to jointly optimize object placement order and goal poses. GaussTwin [5] aims to unify simulation and online tracking of rigid bodies and deformable linear objects such as ropes, combining physics-based prediction with visual correction to maintain the interaction state; SyncTwin [14] targets safe manipulation in dynamic and partially occluded scenes, continuously updating object poses to provide planners with current scene geometry for collision avoidance and grasping. Real-is-Sim [1] uses a synchronized twin to maintain a consistent policy observation interface across training, simulation-based evaluation, and real-world deployment of learned policies. SpatialHarness uses synchronized scenes to provide complementary virtual views and structured spatial information to frozen multimodal policies at runtime, improving spatial observability for precise manipulation.

2.3. Viewpoint Selection for Robotic Manipulation

Prior work improves spatial perception and action prediction in manipulation policies through multi-view rendering or active camera control. RVT [10] renders scene point clouds into predefined virtual views, allowing policies to use viewing directions unconstrained by physical camera placement. RVT-2 [11] further generates zoomed-in local views around the manipulation region predicted by the network to improve localization accuracy in small-scale manipulation. To address occlusion and insufficient detail from fixed viewpoints, ActiveVLA [25] selects candidate views around model-identified regions of interest based on visibility, distance, and view diversity, and combines them with local zooming to improve fine-grained action prediction. TVVE [3] and SaPaVe [24] learn to acquire observations that facilitate manipulation: the former uses reinforcement learning to train a network to predict virtual camera poses from point clouds, optimizing viewpoints with feedback such as action prediction errors to improve coverage of task-relevant information; the latter learns head-camera motions and robot actions from camera-control and manipulation data, enabling the robot to find targets that are occluded or outside its field of view and continue manipulation while adapting to viewpoint changes.

3. Method

3.1. Overview

SpatialHarness is a test-time embodied harness that provides test-time spatial scaffolding for a frozen multimodal policy. Its spatial context comprises complementary virtual views and structured spatial information. At each decision step, the policy receives this spatial context together with the task instruction, real observations, robot states. The policy integrates these inputs to assess the current scene and generate robot actions through tool calls. We use *model* for the same underlying multimodal foundation model when it reasons about constraints, generates constraint programs, or configures virtual viewpoints, and *policy* when it generates actions. The harness executes the programs, advances the simulation, synchronizes the scene, and renders the views. We first describe scene synchronization, followed by task-oriented view selection.

3.2. Interaction-Aware Scene Synchronization

As illustrated in Figure 2, interaction-aware scene synchronization maintains the online simulated scene from which the harness constructs spatial context. Before task execution, the harness aligns the scene with the initial observation. After each action, it synchronizes the scene, providing updated geometry for auxiliary virtual views and object 6-DoF poses. Physics simulation propagates simulation states through physical interactions. To account for errors in physical modeling, visual correction uses real observations to select or replace object poses and replay states, while programmatic correction uses model-generated constraint programs to check or adjust candidates. In the following, p denotes an object pose (position and orientation), whereas s denotes a simulation state that may also contain dynamic information.

3.2.1. Asset and Initial State Setup

The harness first leverages the multimodal model [27] to autonomously construct object CAD models from multiple images of each object and image annotations specifying dimensions of basic geometric elements. Then it establishes their initial poses through visual and programmatic correction. During visual correction, it renders the generated CAD models at candidate poses from the real camera viewpoint using calibrated camera extrinsics. Within valid visible regions, the rendered contours and depth are compared against the corresponding SAM 3 segmentation masks [6] and observed depth in the initial observation o_0. Poses consistent with these observations are retained to form the initial candidate pose set $\mathcal{P}_{i,0}$.

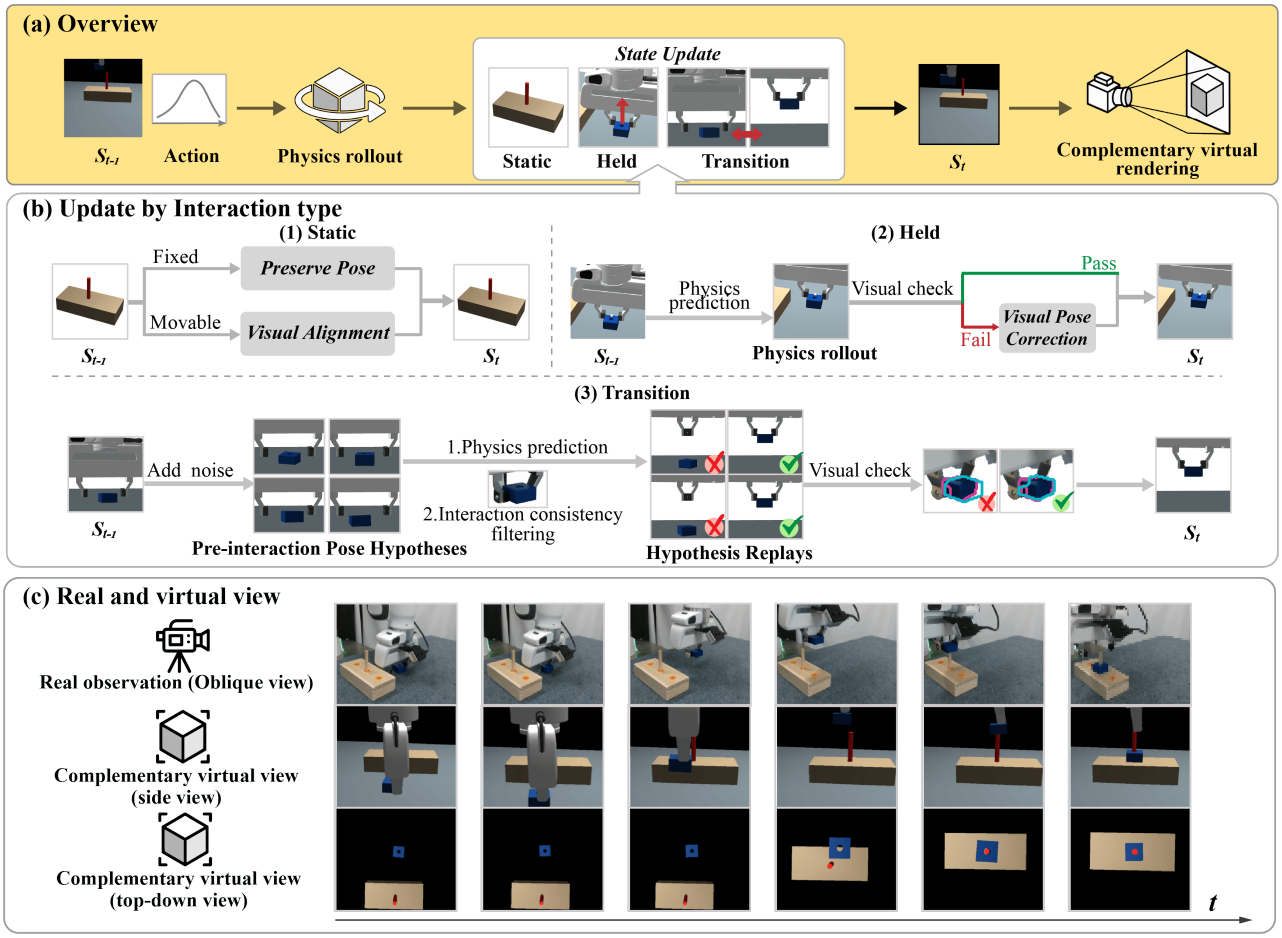


Figure 2. **Online scene synchronization by interaction mode.** (a) Measured robot motion advances the physics simulation, updating the scene state and rendering supplementary views. (b) Stationary object poses are retained or visually aligned according to movability. For continuously grasped objects, poses are predicted through physics simulation and validated; rejected predictions are replaced with visual estimates. When a grasp is established or released, the same measured motion segment is replayed from candidate pre-interaction poses to select the current state by interaction consistency and visual matching. (c) Real oblique and corresponding virtual side and top views of Tower of Hanoi. Columns run chronologically from left to right, showing scene updates during real execution and hole–peg spatial relationships.

An overall visual match does not guarantee accurate alignment of local geometric features. For programmatic correction, the harness provides the model with o_0 and object properties g_i, which describe the shapes, orientations, and mutual relations of these features. Using these properties and scene references identified in o_0, the model infers the required spatial relations, yielding the constraint set $\mathcal{C}_{i,0} = \{c_{i,0}^{(j)}\}_{j=1}^{n_i}$. Here, n_i is the number of applicable constraints, and $c_{i,0}^{(j)}(p) \in \{0,1\}$ tests whether the corresponding relation holds at candidate pose p. For example, a Hanoi peg must lie above the horizontal support surface. The model translates the inferred constraints into a constraint program:

$$z_{i,0} = \pi(\mathcal{C}_{i,0}), \qquad p_{i,0} = \phi(\mathcal{P}_{i,0}, z_{i,0}).$$

Here, π is the model's program-generation operation, and ϕ is the harness's execution of the constraint program. Execution performs constraint checking and may apply bounded pose correction or reject candidates. The harness accepts $p_{i,0}$ only when it satisfies the constraints and passes visual validation. The harness then synchronizes the simulated robot with the measured arm pose and gripper width to complete the scene. Task-relevant features in the simulated scene are visually highlighted, for example, red Hanoi pegs, to clarify spatial relations in auxiliary virtual views.

3.2.2. Scene State Updates

For object i at time t, the harness selects a state update procedure according to $m_{i,t} \in \{\text{static}, \text{held}, \text{transition}\}$.

Static. The harness preserves the poses of immovable objects, with $p_{i,t} = p_{i,t-1}$. For stationary but movable objects, it uses visual correction to update object poses from new observations, followed by programmatic correction and visual validation as in initialization.

Held. From the previously aligned scene, the harness uses measured joint and gripper motion in the physics engine to predict the held object's current pose. The prediction undergoes visual validation and constraint checking against object–gripper geometry, including center-axis offset and feature orientation. If either check fails, wrist-camera-based pose correction replaces the prediction with a visual estimate.

Transition. The harness uses hypothesis replay for state correction across grasp/release transitions. Rather than directly correcting only the post-interaction object pose, it constructs pre-interaction pose hypotheses at a pre-interaction checkpoint, replays the same measured robot motion from each hypothesis, and selects a full current simulation state consistent with real execution. Gripper opening and closing identify the interaction window to be examined. Let τ denote the time of the pre-interaction checkpoint. The harness saves the full simulation state s_τ and records the subsequent measured robot motion $u_{\tau:t}$ and wrist-camera observations $o_{i,\tau:t}$. Local position and orientation perturbations around the object's pose estimate $p_{i,\tau}$ at the checkpoint produce the pre-interaction pose hypotheses $\mathcal{P}_{i,\tau} = \{p_{i,\tau}^{(k)}\}_{k=1}^{n_c}$. Each hypothesis replaces the object's pose in the checkpoint while leaving the rest of that simulation state unchanged, defining a replay state $s_\tau^{(k)}$. Each branch then replays the same measured robot motion:

$$\tilde{s}_t^{(k)} = f\Big(s_\tau^{(k)}, u_{\tau:t}\Big), \qquad k = 1, \dots, n_c.$$

The harness retains both the resulting current state $\tilde{s}_t^{(k)}$ and the state sequence $\tilde{s}_{\tau:t}^{(k)}$ for comparison with real execution. Replay candidate selection proceeds in two stages. First, programmatic correction applies interaction consistency filtering. The model uses the recorded wrist-camera observations and measured robot motion to identify observable changes across the interaction. These changes define interaction consistency constraints $\mathcal{C}_{i,\tau:t}$, from which the model generates a constraint program that the harness executes:

$$z_{i,\tau:t} = \pi(\mathcal{C}_{i,\tau:t}), \qquad b_k = \phi\Big(\tilde{s}_{\tau:t}^{(k)}, z_{i,\tau:t}\Big).$$

Accounting for the wrist camera's own motion, the harness executes the program for constraint checking: it tests whether each simulated grasp or release agrees with the real interaction. Here, ϕ returns a Boolean constraint-checking result rather than a corrected pose. Passing candidates receive $b_k = 1$; the others are filtered out. When at least one replay candidate passes interaction consistency filtering, visual correction uses contour matching to select among the remaining candidates. The harness renders each replay sequence from the wrist camera's viewpoint at the corresponding times and compares object-mask contours across the entire valid observation window:

$$k^\star = \arg\min_{k:\, b_k=1} d_{\mathrm{mask}}\Big(\hat{o}_{i,\tau:t}^{(k)}, o_{i,\tau:t}\Big).$$

Here, $\hat{o}_{i,\tau:t}^{(k)}$ is the observation sequence rendered from candidate k's replay, and d_{mask} measures contour differences between rendered and real object observations. Interaction consistency filtering rejects branches with incorrect grasp or release outcomes, while contour matching distinguishes spatial differences among the remaining branches. The harness accepts the selected candidate's full current simulation state:

$$s_t \leftarrow \tilde{s}_t^{(k^\star)}.$$

This state correction preserves both the object pose and the dynamic state produced by replay, allowing subsequent simulation and auxiliary virtual rendering to continue from the accepted interaction outcome.

3.3. Task-Oriented View Selection

Task-oriented view selection configures virtual viewpoints before execution so that complementary virtual views reveal task-critical spatial relationships. Guided by a multistep reasoning prompt, the model first identifies the criteria for task success from the task instruction and derives the spatial constraints that the relevant objects must satisfy upon task completion. Using object properties extracted from the 3D assets, it determines whether these constraints primarily concern the relative positions and orientations of whole objects or how local structures fit together based on their geometry.

For spatial constraints concerning relations between whole objects, the model selects two virtual viewpoints for global views, with virtual viewpoint selection restricted to specified horizontal and vertical spatial ranges. For spatial constraints involving how local structures fit together, it selects virtual viewpoints for one local view and one global view. The model determines the virtual viewpoint and framing for the local view from the relevant structures' shapes, positions, and orientations, while the global view covers the robot and task-relevant objects. The current real viewing direction serves as a reference for virtual viewpoint selection.

The harness configures virtual cameras at the proposed virtual viewpoints and renders candidate views. The model checks whether the relevant objects or parts are simultaneously visible, whether task-critical spatial relationships can

be clearly assessed, and whether each view reveals information difficult to discern in the current real observation. For unsuitable virtual viewpoints, the model proposes small adjustments to camera position, orientation, and framing, followed by re-rendering and inspection. When necessary, it requests that the harness simulate the motion of relevant objects to check whether the candidate views remain suitable when task-critical spatial relationships need to be assessed. During execution, the virtual viewpoints remain fixed, while the harness performs auxiliary virtual rendering from the continually updated online simulated scene.

4. Experiments

We evaluate SpatialHarness on four real robot tasks to assess its benefits for fine manipulation. With the manipulation model and real camera configuration held fixed, we compare task success rates and model query overhead with and without supplementary spatial observations.

4.1. Experimental Setup

Implementation Details. We use a Franka Research 3 arm and two Intel RealSense RGB-D cameras, one mounted on the wrist and one providing a fixed third-person view. Both methods use Robocurve's Inspect Robots control framework[29] and GPT-6 Astra [27] (low reasoning effort) as the manipulation model. The baseline receives the task instruction, RGB images from both real cameras, measured robot states, and execution feedback. SpatialHarness adds two virtual RGB views and object poses, with the rest of the control pipeline unchanged. Virtual viewpoints are selected by GPT-6 Astra (xhigh reasoning effort) before task execution. MuJoCo [30] is used for physics simulation and image rendering.

Robot Control Protocol. Both methods share the action representation, available action chunk lengths, control frequency, and local execution checks. Each action is a seven-dimensional vector (a=[δx,δy,δz,δr_x,δr_y,δr_z,g]), comprising end-effector translation increments, rotation vector increments, and a gripper command. At each call, the model jointly generates an action chunk and its length $H \in 10, 20, 30$. Before execution, local checks verify output validity and compliance with execution constraints, without additional model calls.

Tasks. Figure 3 presents the four tasks:

- Block stacking requires grasping a block, aligning it above another block, and releasing it into a stable stack.
- Plug insertion requires matching the position and orientation of a plug to its socket before insertion.
- Tower of Hanoi evaluates placement of a holed piece over a target peg, where the policy must assess the relative alignment of the hole and peg.
- Open drawer and place block requires opening an articulated drawer and then placing a block inside.

Together, these tasks range from alignment of mating structures to placement with more permissive final positioning.

Metrics. Each method is evaluated in 15 trials per task. We report task success rate, the fraction of trials that achieve the task objective, and total policy queries, the cumulative number of requests to the manipulation model for action decisions across all trials. The latter measures model interaction overhead.

4.2. Experimental Results

4.2.1. Quantitative Performance

Real-world Experiment Results. As shown in Figure 4, , with the manipulation policy and control protocol held fixed, SpatialHarness improves success rates on all four tasks involving fine manipulation. The average success rate increases from 18.33% to 83.33%, a gain of 65.00 percentage points. The gains are particularly pronounced on the Tower of Hanoi and drawer tasks: the baseline fails in all 15 trials of each task, whereas our method succeeds in 15 and 10 trials, respectively. Under the same real camera configuration, these two tasks depend strongly on auxiliary virtual views that reveal task-critical spatial relationships. The original observations cannot clearly reveal the relative offset between the hole and the peg or the contact relationship between the gripper and the handle, limiting the execution of key alignment and grasping steps. These auxiliary virtual views make these local spatial relationships easier to discern, providing the same manipulation policy with the necessary visual information to complete these steps. Moreover, plug insertion places the greatest demands on precision among the tasks evaluated, requiring precise alignment in both position and orientation. Using the available real observations, the baseline achieves a success rate of 26.67%, indicating that the original views support successful insertion in some trials. SpatialHarness increases the success rate to 66.67%, a gain of 40.00 percentage points. This result highlights another benefit of auxiliary virtual views: even when the original observations already support successful execution, more complete spatial information can still help identify subtle misalignments and substantially improve insertion success. Block stacking follows the same trend, with the success rate increasing from 46.67% to 100%. Together, these two results show that auxiliary virtual views can further improve fine manipulation success on tasks where the baseline already succeeds in some trials.

Decision-call efficiency. As shown in Figure 6, SpatialHarness achieves higher success rates on all four tasks,

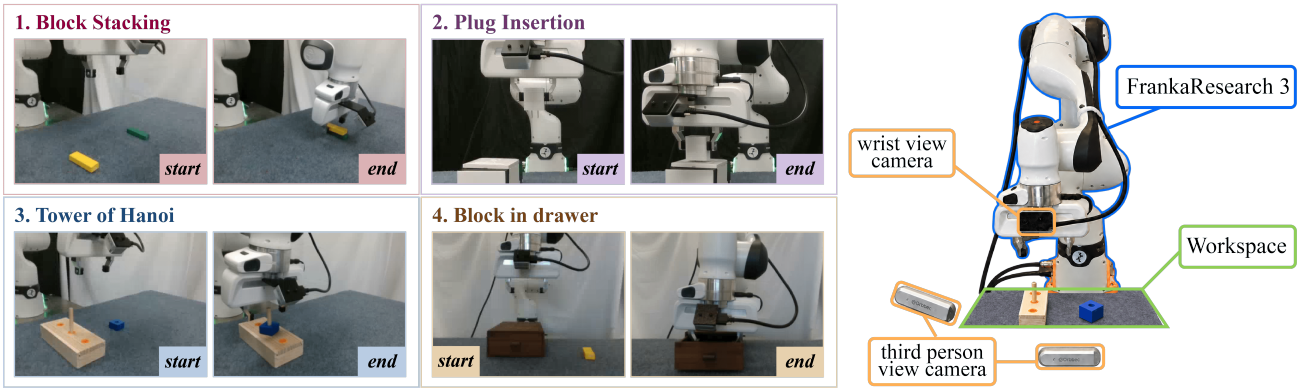


Figure 3. Real robot tasks and experimental platform. Left: initial and final states of block stacking, plug insertion, Tower of Hanoi, and drawer opening followed by block placement. Right: the Franka Research 3 arm, camera positions, and Tower of Hanoi setup.

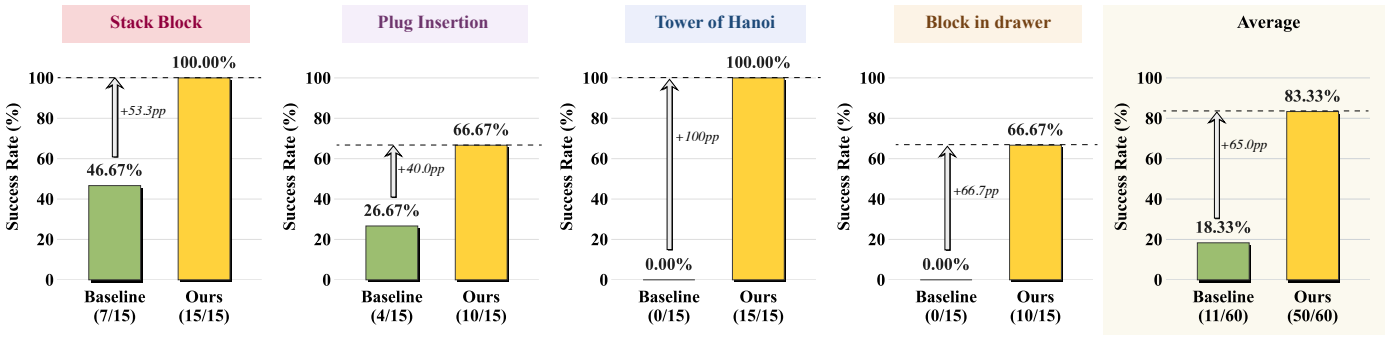


Figure 4. Success rates on real robot tasks. The baseline and SpatialHarness use the same manipulation policy and control protocol, with 15 trials per task and method. The rightmost column gives the average across the four tasks. Parentheses give successes/total trials; arrows show gains over the baseline in percentage points (pp).

while the number of queries to the manipulation policy either decreases or remains largely unchanged. For block stacking and Tower of Hanoi, the number of policy queries decreases by approximately 30% and 21%, respectively, while both tasks achieve a 100% success rate, indicating better task completion with lower policy query overhead. For plug insertion, the success rate increases by 40.00 percentage points, while the number of policy queries still decreases by approximately 7%. For the drawer task, the number of policy queries remains comparable to the baseline, but the success rate rises from 0% to 66.67%, showing that substantially higher task completion rates can be achieved with similar query overhead. Achieving more successes with a comparable or smaller number of policy queries suggests that auxiliary virtual rendering may reduce ineffective attempts and repeated adjustments caused by limited spatial observability, thereby lowering the policy query overhead of fine manipulation.

4.2.2. Qualitative Analysis

Figure 5 compares the baseline and SpatialHarness during real robot execution on four tasks, highlighting behavioral differences in fine alignment, placement, and grasping. In block stacking, the baseline completes grasping and transport, but a placement error prevents a stable stack, while SpatialHarness places the block accurately. Plug insertion requires more precise local alignment: the baseline brings the plug near the socket, but a positional offset from the socket opening persists and insertion fails. SpatialHarness successfully aligns and inserts the plug. In both cases, the differences arise during fine manipulation near the target, illustrating how the spatial context provided by the harness supports accurate placement and insertion. The Tower of Hanoi example further shows that apparent alignment in conventional views can be misleading. With only third-person and wrist views, the hole and peg may appear aligned while their actual spatial offset remains difficult to discern. The baseline moves the block near the target peg but fails to resolve the hole–peg misalignment and cannot fit the block onto it. With auxiliary virtual rendering, SpatialHarness aligns the block and fits it onto the peg, illustrating how auxiliary virtual views reduce spatial ambiguity. The baseline fails at the prerequisite grasp in the drawer task: the gripper approaches the handle but remains misaligned, preventing an effective grasp. The auxiliary virtual views

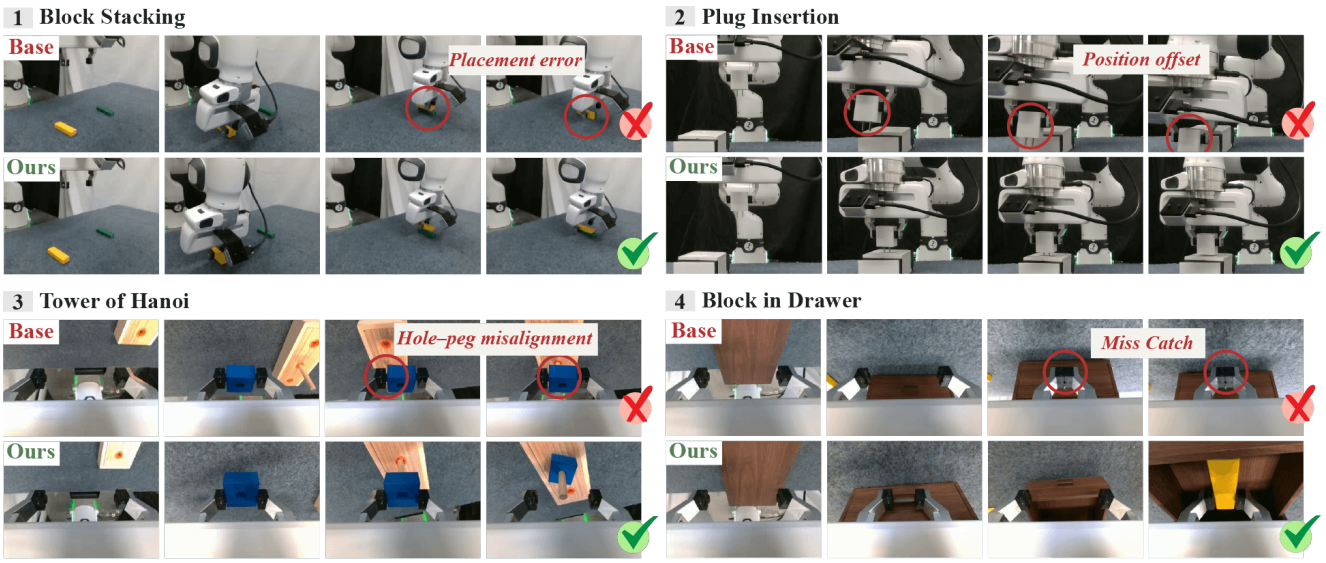


Figure 5. Qualitative comparison of the baseline and SpatialHarness on four real-world manipulation tasks: (1) block stacking, (2) plug insertion, (3) Tower of Hanoi, and (4) block in drawer. Each row shows representative frames from task execution. Red circles highlight failure locations, while crosses and checkmarks indicate task failure and success, respectively.

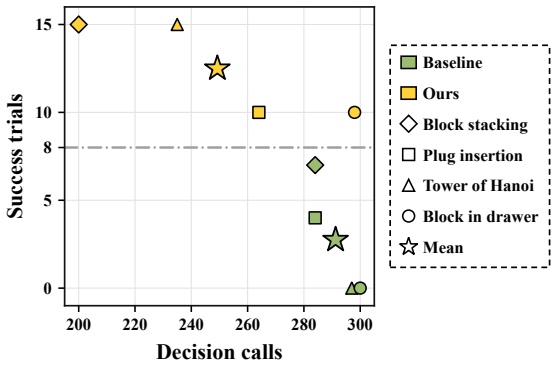


Figure 6. Success trials versus decision calls. SpatialHarness lies above and to the left of the baseline for each plotted task, combining higher success with fewer decision calls. Colors identify methods, marker shapes identify tasks, and stars denote task-average summaries.

clarify the spatial relationship between the gripper and handle, allowing SpatialHarness to align the gripper with the handle and grasp it securely. Together, these cases show that spatial context can improve local alignment and grasping accuracy and reduce errors in spatial judgment from real observations, helping the same policy complete fine manipulation more reliably.

5. Limitations

Our current study focuses on rigid and articulated objects and does not address deformable-object manipulation, where changes in object geometry introduce additional challenges for scene representation and synchronization. Moreover, interaction-aware scene synchronization incurs some time overhead, particularly during grasp/release transitions that require hypothesis replay, interaction consistency filtering, and contour matching, limiting the system's responsiveness. Finally, SpatialHarness currently relies on powerful multimodal foundation models for spatial reasoning and action generation, and its effectiveness with smaller models remains unverified. As multimodal models continue to advance, smaller open-source models may become viable replacements for these frontier models, potentially reducing inference costs and facilitating broader deployment.

6. Conclusion

We presented SpatialHarness, a test-time embodied harness that improves spatial observability for fine robotic manipulation. Through interaction-aware scene synchronization and task-oriented view selection, it provides frozen multimodal policies with spatial context comprising complementary virtual views and structured spatial information. Real-robot experiments show substantially improved manipulation success without policy fine-tuning or changes to physical cameras, highlighting how test-time spatial scaffolding can help realize existing model capabilities.

References

[1] Jad Abou-Chakra, Lingfeng Sun, Krishan Rana, Brandon May, Karl Schmeckpeper, Niko Suenderhauf, Maria Vittoria Minniti, and Laura Herlant. Real-is-sim: Bridging the sim-to-real gap with a dynamic digital twin, 2025.

[2] Anthropic. Claude Opus 5.5, 2026.

[3] Yongjie Bai, Zhouxia Wang, Yang Liu, Kaijun Luo, Yifan Wen, Mingtong Dai, Weixing Chen, Ziliang Chen, Lingbo Liu, Guanbin Li, and Liang Lin. Learning to see and act: Task-aware virtual view exploration for robotic manipulation, 2026.

[4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. In *Proceedings of Robotics: Science and Systems*, LosAngeles, CA, USA, 2025.

[5] Yichen Cai, Paul Jansonnie, Cristiana de Farias, Oleg Arenz, and Jan Peters. Gausstwin: Unified simulation and correction with gaussian splatting for robotic digital twins. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.

[6] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, Rishi Hazra, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollár, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. Sam 3: Segment anything with concepts, 2025.

[7] Yanzhe Chen, Zechen Bai, Zhijun Cao, Wenzheng Zeng, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, Qiming Huang, and Mike Zheng Shou. Show-harness: Just a vlm agent can play robots, 2026.

[8] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.

[9] Dibya Ghosh, Homer Rich Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Quan Vuong, Ted Xiao, Pannag R Sanketi, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An Open-Source Generalist Robot Policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.

[10] Ankit Goyal, Jie Xu, Yijie Guo, Valts Blukis, Yu-Wei Chao, and Dieter Fox. Rvt: Robotic view transformer for 3d object manipulation. In *Proceedings of The 7th Conference on Robot Learning*, pages 694–710. PMLR, 2023.

[11] Ankit Goyal, Valts Blukis, Jie Xu, Yijie Guo, Yu-Wei Chao, and Dieter Fox. Rvt2: Learning precise manipulation from few demonstrations. *RSS*, 2024.

[12] Meng-Hao Guo, Zhe-Han Mo, Jia-Jun Wang, Yi Zhang, Kejin Wang, Yi-Xuan Deng, Jia-Peng Zhang, Yongming Rao, and Shi-Min Hu. Transferring the intelligence of vlms to robotic control, 2026.

[13] Xiaoshen Han, Junqiu Yu, Minghuan Liu, Yilun Chen, Xiaoyang Lyu, Yang Tian, Bolun Wang, Weinan Zhang, and Jiangmiao Pang. Re^{3}sim: Generating high-fidelity simulation data via 3d-photorealistic real-to-sim for robotic manipulation, 2026.

[14] Ruopeng Huang, Boyu Yang, Wenlong Gui, Jeremy Morgan, Erdem Biyik, and Jiachen Li. Synctwin: Fast digital twin construction and synchronization for safe robotic manipulation, 2026.

[15] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025.

[16] Ivan Kapelyukh, Yafei Hu, Ran Gong, Brandon May, Tushar Kusnur, Laura Herlant, Karl Schmeckpeper, Edward Johns, and Xiaohan Zhang. Test-time spatial reasoning for robot manipulation using generative real-to-sim. *arXiv preprint arXiv:2609.33982*, 2026.

[17] Koray Kavukcuoglu. Gemini 4 Argon: Our next era of frontier intelligence, 2026.

[18] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering, 2023.

[19] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In *Proceedings of The 8th Conference on Robot Learning*, pages 2679–2713. PMLR, 2025.

[20] Quentin Le Lidec, Wilson Jallet, Louis Montaut, Ivan Laptev, Cordelia Schmid, and Justin Carpentier. Contact models in robotics: a comparative analysis. *IEEE Transactions on Robotics*, 2024.

[21] Christopher Leet, Achu Menon, Sravanthi Machcha, Sabrina Zou, Aayushya Patel, Aditya Kumar Singh, Anish Kr Singh, Galaba Vamsi, Javin Ahuja, Sai Asish Yamani, Tushar Anand, Vedang Alle, Zihan Jack Zhang, Tzu Kit Chan, and Jay Chooi. Inspect robots: Evaluating the capabilities and safety of embodied ai, 2026.

[22] Bowei Li, Yuner Zhang, and Changliu Liu. Arstag: An agentic real2sim2real system for task-specific robot data generation, 2026.

[23] Fangcheng Liu, Yeqing Shen, Anda Cheng, Weishi Mi, Chao Tang, Chenyuan Liu, Yushun Xiang, Tingguang Li, Yong-Lu Li, and Yehui Tang. Roboicl: Embodied in-context learning with gpt-6 astra, 2026.

[24] Mengzhen Liu, Enshen Zhou, Cheng Chi, Yi Han, Shanyu Rong, Liming Chen, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. Sapave: Towards active perception and

manipulation in vision-language-action models for robotics, 2026.

[25] Zhenyang Liu, Yongchong Gu, Yikai Wang, Xiangyang Xue, and Yanwei Fu. Activevla: Injecting active perception into vision-language-action models for precise 3d robotic manipulation, 2026.

[26] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, pages 405–421, 2020.

[27] OpenAI. GPT-6 Astra: A New Generation of Intelligence. OpenAI, 2026.

[28] Qwen Team. Qwen3.5: Towards native multimodal agents, 2026.

[29] Robocurve. Inspect robots: The open-source evaluation framework for physical ai, 2026.

[30] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012.

[31] Wenbo Zhang, Kaixuan Wang, Yutao Ouyang, Xiaoyu Huang, Liyang Li, Kailun Su, Weiyang Jin, Wenhao Chai, Haotian Liang, Zhiyang Dou, Yue Chen, and Tianxing Chen. An unexpected robot policy: Early evaluations of gpt-6 astra on robodojo and beyond, 2026.

[32] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of The 7th Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.